Draw It or Lose It
CS 230 Project Software Design
Version 3.0

Table of Contents

CS 230 Project Software Design 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 8

Document Revision History

Version	Date	Author	Comments
1.0	01/25/2026	Ava Lindgren	Added information to cover page, executive summary, requirements, design constraints, system architecture view, domain model, evaluation, and recommendations.
2.0	02/08/2026	Ava Lindgren	Updated evaluation and design constraints including going into further detail. Changed recommendation from Windows to Linux as it would be more long-term cost effective, meets the scalability requirements, and it aligns better with web-based architecture. Created a references page for references used in the recommendations.
3.0	02/16/2026	Ava Lindgren	Made final revisions to the document, focusing on the Recommendations section.

Executive Summary

Creative Technology Solutions is working alongside The Gaming Room to design and develop a web-based multi-platform version of their existing game on Android called Draw It or Lose It. The main goal is to make the game accessible on multiple platforms while keeping gameplay consistent for all users. The game will be organized so that teams and players are managed correctly, and game names remain unique. The solution that is proposed ensures that only one game runs in memory at a time in order to avoid conflicts.

Design Constraints

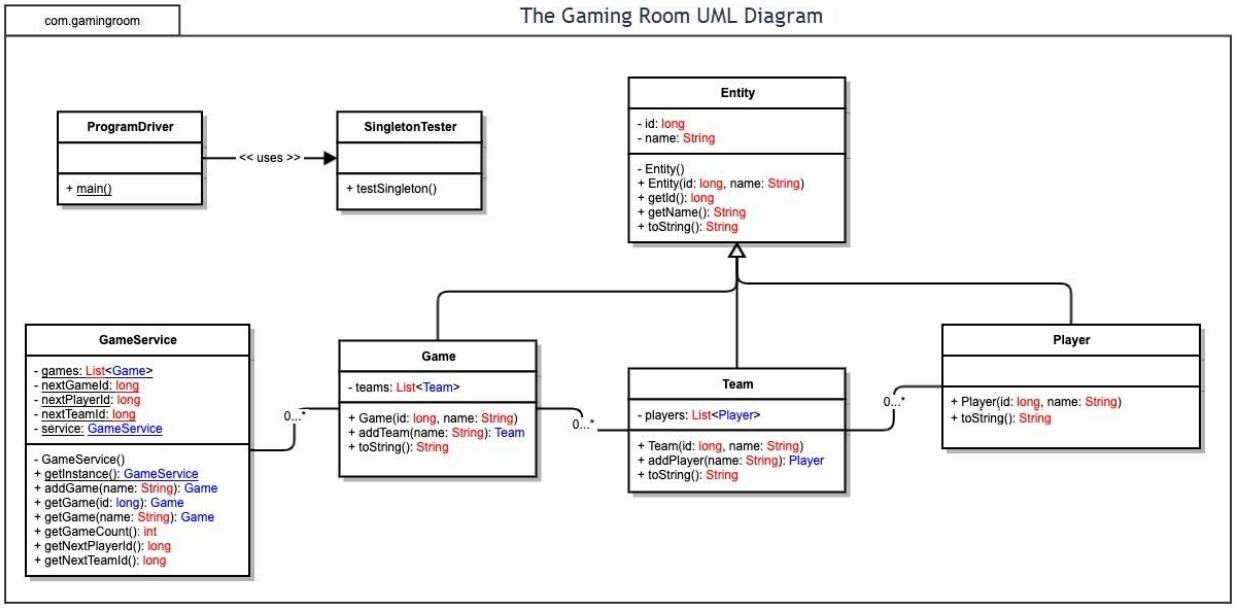
The game must be delivered as a web-based application that supports multiple users connecting simultaneously from different platforms through modern web browsers. The system must manage shared game data in real time while maintaining consistency across all connected players. Because network latency and concurrent user activity can affect performance, the application must be designed to handle synchronization, timing accuracy, and controlled access to a single game instance in memory. These constraints directly influence how game objects are not only created but managed and updated during gameplay. The application must be hosted in a centralized server environment capable of supporting concurrent gameplay and future scalability. It must use widely supported web technologies to ensure cross-platform compatibility without requiring client-side installations. The design must also preserve data integrity and support long-term maintainability.

System Architecture View

The application will follow a basic web structure where users access the game through a web browser and interact with a central server. The server will manage game logic, teams, and the players.

Domain Model

The UML class diagram shows the main classes used in the game and their relations with one another. The class, GameService, is the central manager for the application and maintains a list of all games. It uses the singleton pattern to make sure only one instance of the service exists in the memory, which supports the requirement of only one game instance being active at a time. The ProgramDriver and SingletonTester classes are used to begin the application and to verify the singleton behavior is working right. The Game class contains a list of Team objects; each Team contains a list of Player objects which is one-to-many relationships in the model. The methods addGame, addTeam, and addPlayer are used to create and manage the relationships. Additionally, Game, Team, and Player classes all inherit from the entity base class that provides shared attributes like a unique name and ID. Object-oriented principles are involved all throughout the design as inheritance and encapsulation and reuse, and they allow the application to meet the requirements for the client.


com.gamingroom
The Gaming Room UML Diagram
ProgramDriver
+ main()
<< uses >>
SingletonTester
+ testSingleton()
Entity
- id: long
- name: String
- Entity()
+ Entity(id: long, name: String)
+ getId(): long
+ getName(): String
+ toString(): String
GameService
- games: List<Game>
- nextGameId: long
- nextPlayerId: long
- nextTeamId: long
- service: GameService
- GameService()
+ getInstance(): GameService
+ addGame(name: String): Game
+ getGame(id: long): Game
+ getGame(name: String): Game
+ getGameCount(): int
+ getNextPlayerId(): long
+ getNextTeamId(): long
0...*
Game
- teams: List<Team>
+ Game(id: long, name: String)
+ addTeam(name: String): Team
+ toString(): String
0...*
Team
- players: List<Player>
+ Team(id: long, name: String)
+ addPlayer(name: String): Player
+ toString(): String
0...*
Player
+ Player(id: long, name: String)
+ toString(): String

Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can host web applications using its Unix-based architecture and built-in web services. However, hardware costs are high, and scalability is limited compared to Linux and Windows. macOS is not commonly used for large-scale production servers, as Apple has discontinued macOS Server and requires Apple hardware. As a result, macOS is best suited for development or testing environments rather than hosting a web application expected to scale to thousands of users.	Linux is the most widely used platform for hosting web-based applications due to its stability, performance, and scalability. It fully supports server-based deployment and integrates well with cloud and container platforms. Most Linux distributions are open source, resulting in minimal licensing costs and making Linux a strong choice for large-scale and cost-effective hosting.	Windows supports robust server-based deployment through Windows Server and Internet Information Services (IIS). It provides enterprise-grade reliability, strong security features, and seamless integration with Microsoft cloud services such as Azure. Windows Server scales effectively for high-traffic web applications, though licensing and resource requirements increase overall operating costs compared to Linux.	Mobile devices are not suitable for server-side hosting due to limitations in performance, availability, and scalability. They do not support continuous server operation and should only be considered for client-side access to the web application.

Client Side	Mac clients are supported through modern web browsers such as Safari and Google Chrome. Because the application is delivered as a responsive HTML interface, no platform-specific client development is required. However, cross-browser testing, particularly for Safari, requires access to Apple hardware, which moderately increases development time and cost.	Linux clients access the application through standards-compliant web browsers. A browser-based architecture allows the same codebase to be used across platforms, keeping development and maintenance costs low. While additional testing across distributions may be required, client support is generally manageable.	Windows has the largest desktop user base, making it a highly accessible client platform. The browser-based delivery model ensures compatibility across Windows devices using Edge, Chrome, and Firefox. Strong operating system stability and consistent browser behavior help reduce client-side support issues, though cross-browser testing is still required.	Mobile clients require responsive, touch-friendly web interfaces to support both iOS and Android platforms. Mobile access is provided through web browsers rather than native applications. While device and browser testing increase development time, this approach enables broad accessibility without maintaining separate mobile codebases.

Development Tools	Mac supports standard web technologies including HTML, CSS, and JavaScript, with tools such as VS Code, PyCharm, and GitHub. These tools are largely cross-platform and low-cost, allowing a single development team to support Mac clients without additional licensing or staffing requirements.	Linux supports common programming languages such as JavaScript, Java, and Python, along with tools including VS Code, Eclipse, Apache, Node.js, and Nginx. These tools are primarily open source, minimizing licensing costs and enabling a single development team to manage both development and server deployment efficiently.	Windows offers a mature development ecosystem, including tools such as Visual Studio, VS Code, Eclipse, PyCharm, and C# with the .NET framework. Many of these tools are free or open source, and Windows provides strong vendor support and long-term maintenance options, reducing onboarding time and operational risk for development teams.	Mobile client support relies on standard web technologies such as HTML, CSS, and JavaScript, using common development tools like VS Code and browser-based testing tools. This shared technology stack minimizes licensing costs and allows a single development team to support both mobile and desktop clients.

Recommendations

1. Operating Platform: Linux is recommended as the server operating platform for Draw It or Lose It due to its stability, performance efficiency, and suitability for hosting web-based applications. Linux is widely used for web servers and cloud environments, offering a lower total cost of ownership by eliminating operating system licensing fees whilst supporting large-scale deployments (Whittaker, 2024). This makes it very well suited for The Gaming Room's needs to expand the application across multiple environments with reliable and scalable hosting.

2. Operating Systems Architectures**:** Linux uses a modular and layered architecture that supports kernel, system services, as well as user applications. This design improves system stability and performance on top of security (Day, 2023). Linux supports both physical and virtualized environments which make it ideal for scalable web deployment and cloud-based hosting where resources can be adjusted as user demand grows.

3. Storage Management: Linux supports a wide range of storage management systems including relational databases, like MySQL and PostgreSQL, which are commonly used for managing game state and user data. These systems provide data integrity, concurrency control, and allow multiple users to have reliable access to it. Linux file systems are designed for performance and reliability, supporting continuous server operation (GeeksforGeeks, 2026).

4. Memory Management: Linux uses virtual memory, paging, and process isolation which are advanced memory management techniques that efficiently allocate system resources (Yogesh, 2024). These mechanisms allow the Draw It or Lose It application to handle multiple concurrent players and sessions while maintaining consistent performance and preventing individual processes from impacting stability.

5. Distributed Systems and Networks: Draw It or Lose It can be implemented using a client-server architecture hosted on a Linux server with clients communicating through HTTP/ HTTPS. Linux is well suited for distributed systems, and it supports load balancing and redundancy. This helps manage network connectivity and outages, and it maintains reliable communication between the server and clients across multiple platforms (Day, 2023).

6. Security: Linux provides strong security features, and data can be protected through encryption for both stored information and network communication (Whittaker, 2024). Some of the security features include user permissions, firewalls, access controls, and secure process isolation. Regular updates and a large open-source security community help identify and resolve vulnerabilities quickly ensuring that user information remains protected across all supported platforms.

References

Day, B. (2023, August 7). *Building Scalable Software Architectures on Linux for Security and Growth*. Linux Security. https://linuxsecurity.com/features/navigating-software-scalability

GeeksforGeeks. (2026, January 12). *Linux File System*. GeeksforGeeks. https://www.geeksforgeeks.org/linux-unix/linux-file-system/

Whittaker, G. (2024, September 10). How Linux Shapes Modern Cloud Computing. *Linux Journal*. https://www.linuxjournal.com/content/how-linux-shapes-modern-cloud-computing

Yogesh. (2024, November 15). *Linux — Memory Management*. Medium. https://lambdafunc.medium.com/linux-memory-management-2c13b5e386f1